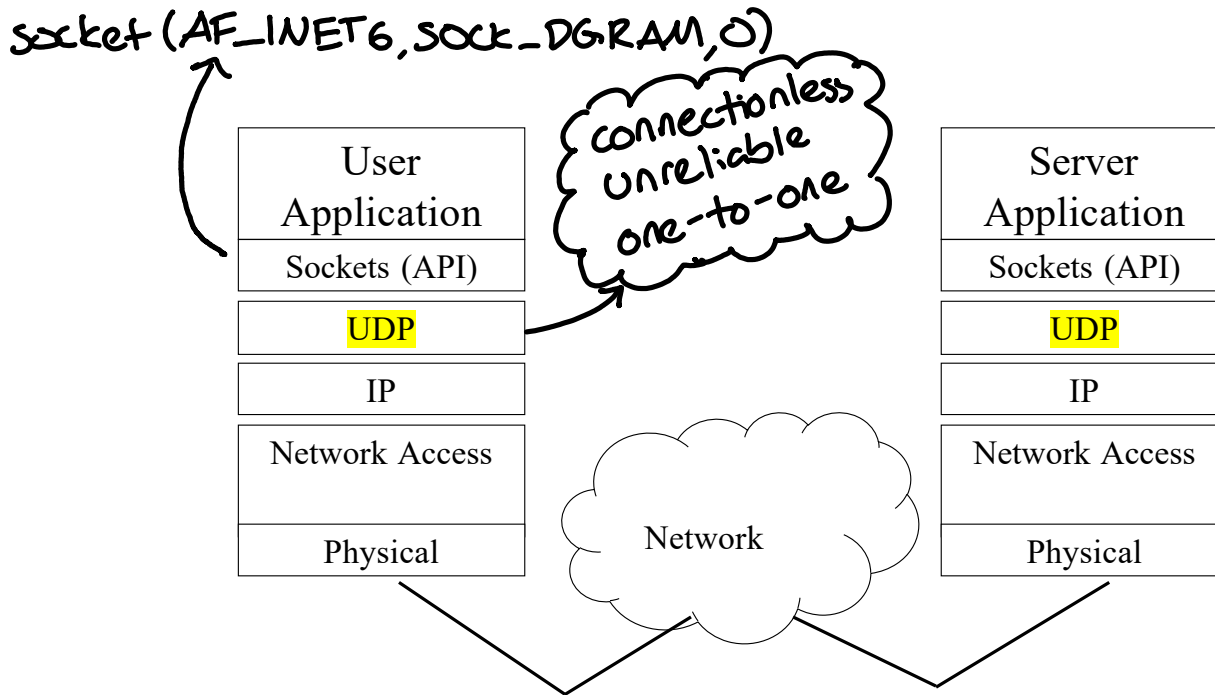




1

Types of Sockets – TCP and UDP

- **User Datagram Protocol (UDP)**
 - Is a **connectionless** service.
 - UDP is a **best effort** service (drop it on the network and forget about it)
 - It uses **individual message transfers** to send data.
 - Socket Programming: **SOCK_DGRAM**

`tcp : SOCK_STREAM`

```
if ((socketNumber = socket(AF_INET6, SOCK_DGRAM, 0)) < 0)
{
    perror("socket call");
    exit(-1);
}
```

2

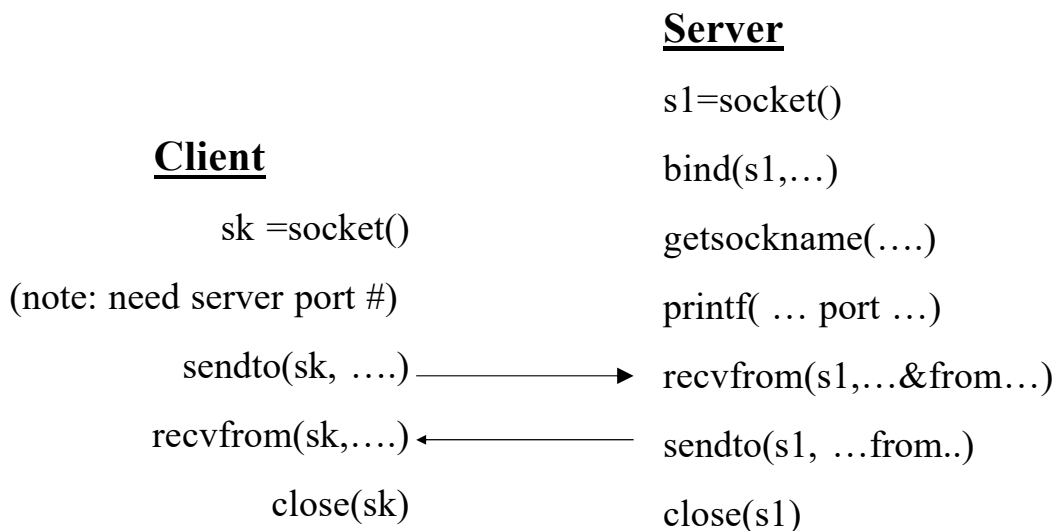
socket() System Call

```
int socket(address_family, type, protocol);
```

- **Address Family**
 - **AF_INET6**
 - Supports both IPv6 and IPv4
 - Use AF_INET4 for IPv4 only
- **Type of communications**
 - TCP use **SOCK_STREAM**
 - **UDP use SOCK_DGRAM**
- **Protocol = 0** (means use default)

3

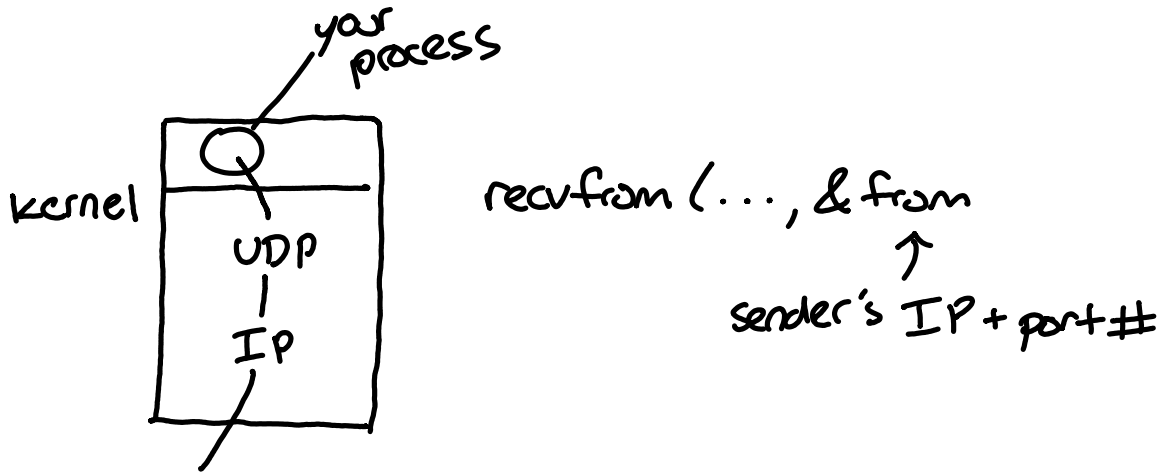
A UDP connection



**The server picks up the client info (client IP and Port)
via the 'from' paramater in the recvfrom() call**

4

Server with Multiple Clients



- UDP has less system calls (than TCP) but can error without notifying
- client/server can't know if other side closes
↳ uses poll with a time value

5

UDP Sending and Receiving Data

- UDP Receive data:

```
int recvfrom(int socket_num, char *data, int data_len, int flags,  
             (struct sockaddr *) &from_address, (size_t *) &from_len)
```

load by kernel with
sender IP and port #
- UDP Send data

```
int sendto(int socket_num, char *data, int data_len, int flags,  
           (struct sockaddr *) &to_address, sizeof(sockaddr));
```

to_address contains IP and port # of destination

 - IP and port #

- Both functions return the amount of data sent/received.
- If return value is < 0 then an error occurred.

6

A sample run udp_server and udp_client

UDP Server Output

```
husmith@unix5:~/cpe464/udp_code$ ./udp_server 55554
Server using Port #: 55554
Client info - IP: ::ffff:129.65.128.85 Port: 36623 Len: 6 hello
Client info - IP: ::ffff:129.65.128.85 Port: 36623 Len: 3 ok
Client info - IP: ::ffff:129.65.128.85 Port: 36623 Len: 7 enough
Client info - IP: ::ffff:129.65.128.81 Port: 45275 Len: 11 aNewClient
Client info - IP: ::ffff:129.65.128.81 Port: 45275 Len: 13 Client2Again
```

UDP Client Output (first client)

```
husmith@unix5:~/cpe464/udp_code$ ./udp_client unix5.csc.calpoly.edu 55554
Server info - IP: ::ffff:129.65.128.85 Port: 55554
Enter the info to transmit: hello
Sending: hello with len: 6
Returned from server "bytes: 6"
Enter the info to transmit: ok
Sending: ok with len: 3
Returned from server "bytes: 3"
Enter the info to transmit: enough
Sending: enough with len: 7
Returned from server "bytes: 7"
```